

SFWE415 – TERM PROJECT (Spring2026)

Multi-Language Microservices Platform

Create a Multi-Language E-Commerce Platform using microservices architecture. This platform will consist of several independent services, each developed in different programming languages, that collaboratively provide a seamless e-commerce experience.

Project Objectives

- Understand and implement microservices architecture.
- Develop services using different programming languages.
- Ensure seamless communication between heterogeneous services.
- Deploy and manage services using containerization tools.
- Implement API gateways and handle authentication/authorization.
- Apply best practices in documentation, testing, and version control.

Project Components

1. User Service

- Language: JavaScript (Node.js)
- Responsibilities:
 - User registration, login, and authentication.
 - Manage user profiles and preferences.
- Technologies:
 - Express.js for building RESTful APIs.
 - JWT (JSON Web Tokens) for authentication.
 - MongoDB for data storage.

2. Product Service

- Language: Python
- Responsibilities:
 - Manage product listings, categories, and inventories.
 - Handle product search and filtering.
- Technologies:
 - Flask or FastAPI for building RESTful APIs.
 - PostgreSQL for data storage.
 - Elasticsearch for search functionality.

3. Order Service

- Language: Java
- Responsibilities:
 - Process customer orders and manage order history.
 - Handle payment processing and invoicing.
- Technologies:
 - Spring Boot for building RESTful APIs.
 - MySQL for data storage.
 - Integration with payment gateways (e.g., Stripe API).

4. API Gateway

- Language: TypeScript (Node.js with Express or NestJS)
- Responsibilities:
 - Serve as a single-entry point for all client requests.
 - Route requests to appropriate microservices.
 - Handle cross-cutting concerns like authentication, logging, and rate limiting.
- Technologies:
 - Express.js or NestJS.
 - Kong or NGINX as alternatives for API Gateway solutions.

5. Frontend Application

- Language: JavaScript (React)
- Responsibilities:
 - Provide a user-friendly interface for interacting with the platform.
 - Communicate with the API Gateway to perform operations like browsing products, managing the cart, and placing orders.
- Technologies:
 - React for building the user interface.
 - Axios or Fetch API for making HTTP requests.

Integration and Communication

- Inter-Service Communication: Utilize RESTful APIs for synchronous communication between services. For asynchronous tasks (e.g., sending notifications after an order is placed), use message brokers like RabbitMQ or Apache Kafka.

- Service Discovery: Implement service discovery mechanisms using tools like Consul or Eureka to allow services to find and communicate with each other dynamically.
- Configuration Management: Use environment variables and configuration files managed through tools like Spring Cloud Config or Consul.

Deployment and Containerization

- Docker: Containerize each microservice to ensure consistency across different development and production environments.
- Docker Compose: For local development, use Docker Compose to orchestrate multi-container applications.

Additional Requirements

1. Version Control
 - Use Git and platforms like GitHub or GitLab for source code management and collaboration.
2. Continuous Integration/Continuous Deployment (CI/CD)
 - Set up CI/CD pipelines using tools like Jenkins, Travis CI, or GitHub Actions to automate testing and deployment.
3. Testing
 - Implement unit tests, integration tests, and end-to-end tests for each service using appropriate testing frameworks (e.g., Jest for Node.js, PyTest for Python, JUnit for Java).
4. Documentation
 - Document APIs using Swagger/OpenAPI.
 - Maintain clear README files and project documentation outlining setup instructions, architecture diagrams, and usage guides.
5. Security
 - Implement secure authentication and authorization mechanisms.
 - Ensure data protection and secure communication between services (e.g., using HTTPS).
6. Monitoring and Logging
 - Set up centralized logging using tools like ELK Stack (Elasticsearch, Logstash, Kibana) or Prometheus & Grafana for monitoring service health and performance.

Project Timeline (Suggested)

1. Week 1-2:
 - Project Planning: Define requirements, assign roles, and design the overall architecture.
 - Set Up Version Control: Initialize repositories and establish branching strategies.
2. Week 3-4:
 - Develop User Service and Product Service: Implement basic CRUD operations and APIs.
 - Set Up API Gateway: Configure routing to the initial services.
3. Week 5-6:
 - Develop Order Service and Inventory Service: Ensure integration with User and Product services.
 - Implement Authentication and Authorization: Secure the APIs via the API Gateway.
4. Week 7-8:
 - Frontend Development: Start building the React frontend and connect it to the API Gateway.
5. Week 9-10:
 - Testing: Perform unit, integration, and end-to-end testing.
 - Documentation: Complete API documentation and user guides.
6. Week 11-12:
 - Deployment: Containerize services using Docker and deploy locally or on a cloud platform.
 - Final Touches: Optimize performance, fix bugs, and prepare for presentation/demo.

Suggested Technologies and Tools

- Programming Languages: JavaScript (Node.js), Python, Java, TypeScript, React.
- Frameworks: Express.js, Flask/FastAPI, Spring Boot, Gin, NestJS, React.
- Databases: MongoDB, PostgreSQL, MySQL, Redis.
- APIs: RESTful APIs, Swagger/OpenAPI for documentation.
- Containerization: Docker, Docker Compose.
- Orchestration: Kubernetes (optional).
- CI/CD: Jenkins, GitHub Actions, Travis CI.
- Version Control: Git, GitHub/GitLab.
- Messaging: RabbitMQ, Apache Kafka.
- Monitoring: ELK Stack, Prometheus, Grafana.

Learning Outcomes

By completing this project, you will:

- Gain practical experience in building and managing microservices.
- Learn to work with multiple programming languages and integrate diverse technologies.
- Understand the complexities of inter-service communication and data consistency.
- Develop skills in containerization and deploying distributed systems.
- Enhance their ability to collaborate using version control and CI/CD pipelines.
- Apply best practices in software development, including testing, documentation, and security.

Final Deliverables

- **Source Code:** Complete codebase for all microservices and the frontend application, hosted on a version control platform.
- **Documentation:** Comprehensive documentation covering architecture, APIs, setup instructions, and usage guides.
- **Deployment:** Deployed application accessible locally or on a cloud platform with instructions.
- **Presentation/Demo:** A demonstration showcasing the functionalities, architecture, and integration of the microservices platform.

Deadline: 1st June, 2026.